labsheet =05 computer vision Priyanshu Sharma Cu240250980 section=(B) (47)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import exposure
```

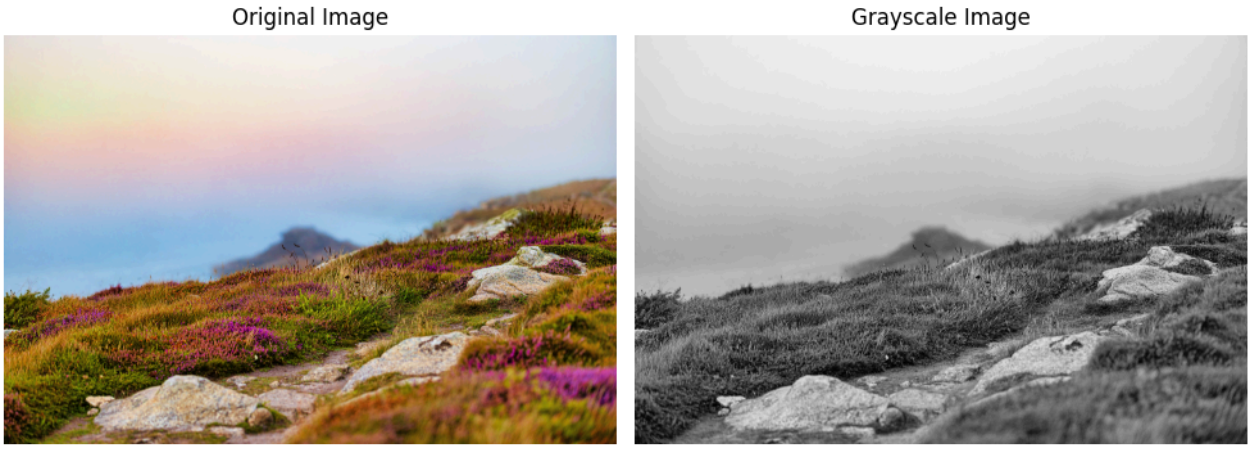
```
# EXPERIMENT 5
# Feature Extraction and Image Analysis using SIFT and HOG
# ============================================================

print("=" * 60)
print("EXPERIMENT 5")
print("Feature Extraction using SIFT and HOG")
print("=" * 60)
```

```
============================================================
EXPERIMENT 5
Feature Extraction using SIFT and HOG
============================================================
```

```
# 1. LOAD IMAGE
# ============================================================

# Use r before the path to avoid Windows backslash problems
image_path = "/content/pexels-friededia-30649280.jpg"

# IMPORTANT: pass image_path inside cv2.imread()
image = cv2.imread(image_path)

if image is None:
    print("\nERROR: Image not found!")
    print("Please check that the image path is correct.")
    print("Path used:")
    print(image_path)
    exit()

print("\nImage loaded successfully!")

# Convert BGR to RGB for Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print("Image size:", image.shape)
print("Grayscale conversion completed.")
```

```
Image loaded successfully!
Image size: (5464, 8192, 3)
Grayscale conversion completed.
```

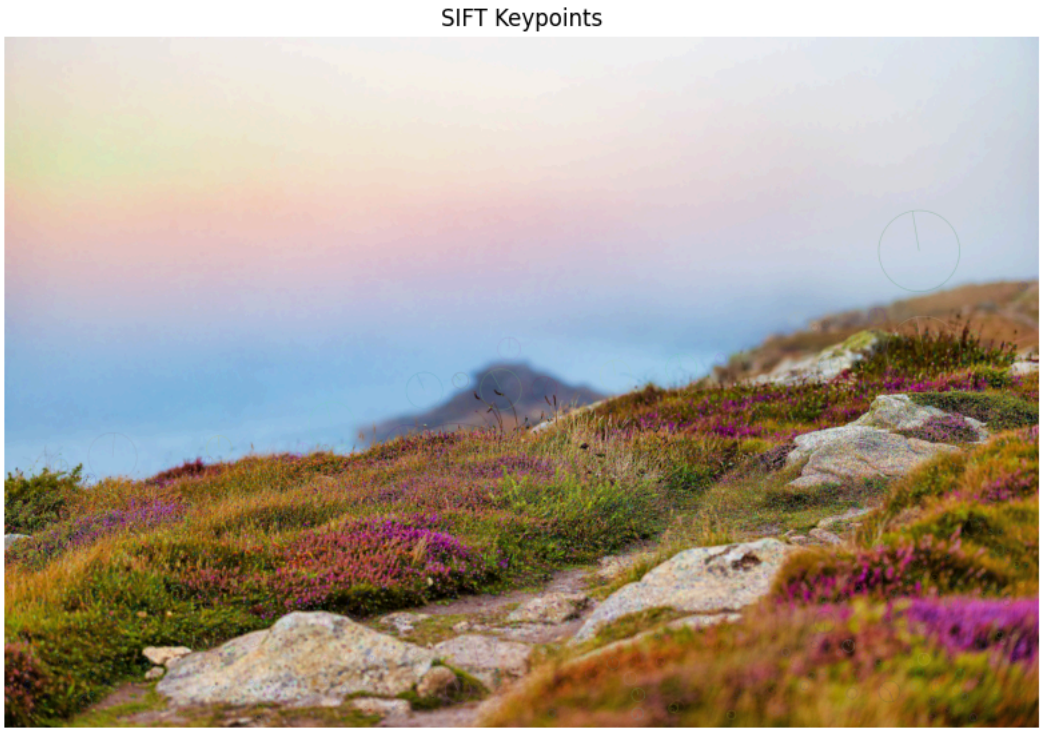
```
# 2. DISPLAY ORIGINAL AND GRAYSCALE IMAGE
# ============================================================

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
# 3. SIFT FEATURE EXTRACTION
# ============================================================

print("\n" + "=" * 60)
print("SIFT FEATURE EXTRACTION")
print("=" * 60)

# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and descriptors
keypoints, descriptors = sift.detectAndCompute(gray, None)

print("Number of SIFT keypoints:", len(keypoints))

if descriptors is not None:
    print("SIFT descriptor shape:", descriptors.shape)
else:
    print("No SIFT descriptors found.")
```

```
============================================================
SIFT FEATURE EXTRACTION
============================================================
Number of SIFT keypoints: 121295
SIFT descriptor shape: (121295, 128)
```

```
# 4. VISUALIZE SIFT KEYPOINTS
# ============================================================

sift_image = cv2.drawKeypoints(
    image_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(10, 7))
plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis("off")
plt.show()
```

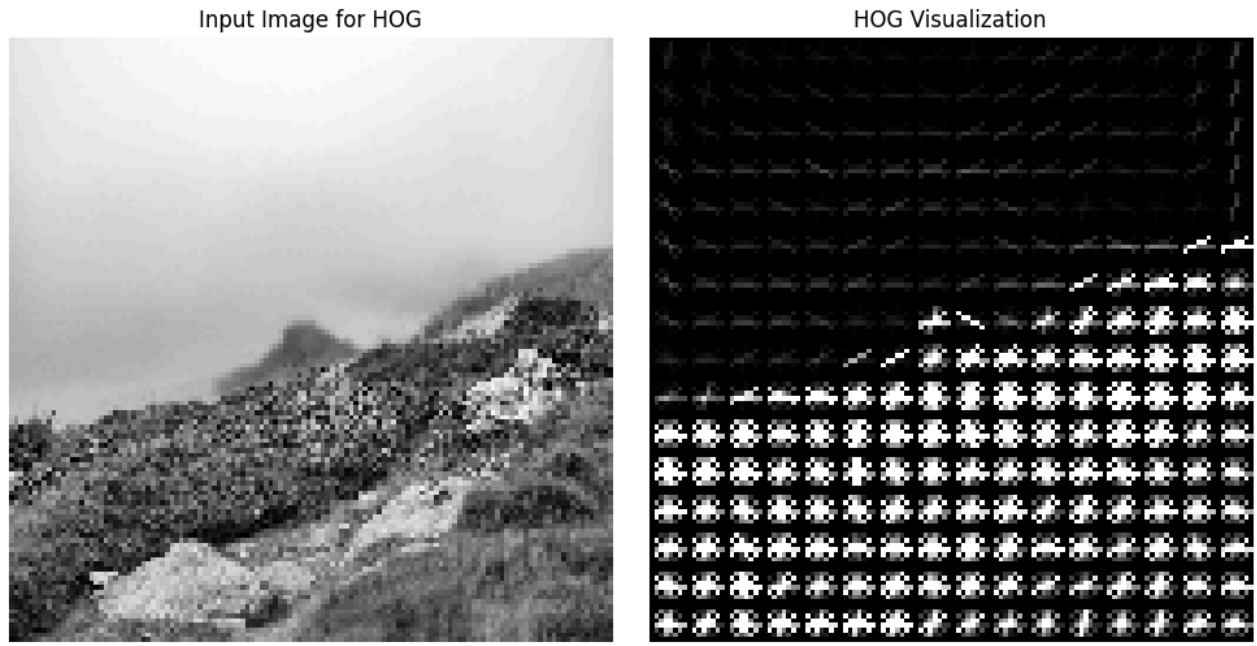



```
# 5. HOG FEATURE EXTRACTION
# ============================================================

print("\n" + "=" * 60)
print("HOG FEATURE EXTRACTION")
print("=" * 60)

# Resize image for HOG
hog_image_input = cv2.resize(gray, (128, 128))

# Extract HOG features
hog_features, hog_image = hog(
    hog_image_input,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm="L2-Hys",
    visualize=True
)

print("Number of HOG features:", len(hog_features))
print("HOG feature extraction completed.")
```

```
============================================================
HOG FEATURE EXTRACTION
============================================================
Number of HOG features: 8100
HOG feature extraction completed.
```

```
# 6. VISUALIZE HOG
# ============================================================

hog_image_rescaled = exposure.rescale_intensity(
    hog_image,
    in_range=(0, 10)
)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(hog_image_input, cmap="gray")
plt.title("Input Image for HOG")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_image_rescaled, cmap="gray")
plt.title("HOG Visualization")
plt.axis("off")

plt.tight_layout()
```

```
plt.show()
```




```
# 7. DISPLAY FEATURE INFORMATION
# ============================================================

print("\n" + "=" * 60)
print("FEATURE INFORMATION")
print("=" * 60)
print("PRIYANSHU SHARMA CU240250980")

print("\nSIFT:")
print("Keypoints detected:", len(keypoints))

if descriptors is not None:
    print("Descriptor dimensions:", descriptors.shape)

print("\nHOG:")
print("Feature vector length:", len(hog_features))
print("Orientations:", 9)
print("Pixels per cell:", (8, 8))
print("Cells per block:", (2, 2))
```

```
============================================================
FEATURE INFORMATION
============================================================
PRIYANSHU SHARMA CU240250980

SIFT:
Keypoints detected: 121295
Descriptor dimensions: (121295, 128)

HOG:
Feature vector length: 8100
Orientations: 9
Pixels per cell: (8, 8)
Cells per block: (2, 2)
```

```
# 8. BASIC IMAGE MATCHING USING SIFT
# ============================================================

print("\n" + "=" * 60)
print("SIFT IMAGE MATCHING")
print("=" * 60)

# Create second image by resizing the original
image2 = cv2.resize(
    image,
    None,
    fx=0.9,
    fy=0.9,
    interpolation=cv2.INTER_LINEAR
)
```

```
# Convert second image to grayscale
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

# Detect SIFT features in second image
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)

print("Keypoints in Image 1:", len(keypoints))
print("Keypoints in Image 2:", len(keypoints2))
```

```
============================================================
SIFT IMAGE MATCHING
============================================================
Keypoints in Image 1: 121295
Keypoints in Image 2: 107428
```

```
# 9. MATCH SIFT FEATURES
# ============================================================

if descriptors is not None and descriptors2 is not None:

    # FLANN matcher
    index_params = dict(
        algorithm=1,
        trees=5
    )

    search_params = dict(
        checks=50
    )

    flann = cv2.FlannBasedMatcher(
        index_params,
        search_params
    )

    # Find two nearest matches
    matches = flann.knnMatch(
        descriptors,
        descriptors2,
        k=2
    )

    # Lowe's ratio test
    good_matches = []

    for pair in matches:

        if len(pair) == 2:

            m, n = pair

            if m.distance < 0.7 * n.distance:
                good_matches.append(m)

    print("Good matches found:", len(good_matches))
```

```
Good matches found: 70587
```

```
# ============================================================
# 10. DISPLAY MATCHING RESULT
# ============================================================

if descriptors is not None and descriptors2 is not None:

    image2_rgb = cv2.cvtColor(image2, cv2.COLOR_BGR2RGB)

    # Make sure matches are valid
    valid_matches = []

    for match in good_matches:
        if match.queryIdx < len(keypoints) and match.trainIdx < len(keypoints2):
            valid_matches.append(match)

    print("Valid matches:", len(valid_matches))

    if len(valid_matches) > 0:

        matching_result = cv2.drawMatches(
```

```
            image_rgb,
            keypoints,
            image2_rgb,
            keypoints2,
            valid_matches,
            None,
            flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
        )

        plt.figure(figsize=(15, 7))
        plt.imshow(matching_result)
        plt.title("SIFT Image Matching")
        plt.axis("off")
        plt.tight_layout()
        plt.show()

    else:
        print("No valid matches found.")

else:
    print("Image matching could not be performed.")
```

```
Valid matches: 70587
```


```
# 11. FINAL COMPARISON
# ============================================================

print("\n" + "=" * 60)
print("SIFT vs HOG COMPARISON")
print("=" * 60)

print("""
SIFT:
- Detects important keypoints in an image.
- Provides feature descriptors for keypoints.
- Robust to scale changes.
- Robust to rotation changes.
- Useful for image matching and object recognition.
- Usually computationally more expensive than HOG.

HOG:
- Represents an image using gradient orientations.
- Captures shape and edge information.
- Commonly used for object detection.
- Computationally simpler than SIFT in many applications.
- Less suitable for rotation changes.
- Depends on parameters such as cell size and block size.
""")
```

```
============================================================
SIFT vs HOG COMPARISON
============================================================

SIFT:
- Detects important keypoints in an image.
- Provides feature descriptors for keypoints.
- Robust to scale changes.
- Robust to rotation changes.
- Useful for image matching and object recognition.
- Usually computationally more expensive than HOG.

HOG:
```

```
- Represents an image using gradient orientations.
- Captures shape and edge information.
- Commonly used for object detection.
- Computationally simpler than SIFT in many applications.
- Less suitable for rotation changes.
- Depends on parameters such as cell size and block size.
```

```
# 12. OBSERVATION
# ============================================================

print("\n" + "=" * 60)
print("OBSERVATION")
print("=" * 60)

print("""
SIFT successfully detected distinctive keypoints and generated
descriptors from the image.

HOG successfully extracted gradient-based features representing
the shape and edge structure of the image.

SIFT is suitable for image matching and recognition where
scale and rotation changes may occur.

HOG is suitable for shape-based object detection and
classification tasks.

Therefore, both techniques are useful handcrafted feature
extraction methods, but they are suitable for different
computer vision applications.
""")

print("=" * 60)
print("EXPERIMENT COMPLETED SUCCESSFULLY")
print("=" * 60)
```

```
============================================================
OBSERVATION
============================================================

SIFT successfully detected distinctive keypoints and generated
descriptors from the image.

HOG successfully extracted gradient-based features representing
the shape and edge structure of the image.

SIFT is suitable for image matching and recognition where
scale and rotation changes may occur.

HOG is suitable for shape-based object detection and
classification tasks.

Therefore, both techniques are useful handcrafted feature
extraction methods, but they are suitable for different
computer vision applications.
```